連立一次方程式における直接解法と反復解法の比較研究

BV25089 村上明日菜

2026 年 5 月 16 日

目次

1 **はじめに** 2

2 **連立一次方程式とは** 2

3 **直接解法** 3

3.1 ガウスの消去法 3

3.2 連立１次方程式をガウスの消去法で解くプログラム 6

3.3 ガウスの消去法の計算量 9

4 **反復解法** 10

4.1 ヤコビ法 10

4.2 連立１次方程式をヤコビ法で解くプログラム 11

4.3 ヤコビ法の計算量 15

5 **直接解法・反復解法の比較評価** 15

6 **最後に** 16

7 **参考文献** 16

1　はじめに

数値解析とは, 数学の問題をコンピュータで解ける形式に翻訳し近似的に答えを出す技術のことである.

数値解析は感染症の広がり, 金融, 機械学習, 物理シミュレーションなど様々な分野の基盤技術として活用されている. 他にも, 身近なところでは, 飛行機が飛ぶ際や天気予報に対し, 物理方程式を用いてコンピュータに計算させ大気の状態をシミュレーションしたりと, 私たちの生活を支える上で不可欠だ.

しかし, 数値解析には常に「誤差」という課題がつきまとう. 計算で誤差が生じてしまうと, 初期値のわずかなずれのせいで最終結果が大きく変わってしまう・丸め誤差が蓄積されシミュレーションが長時間になるほど精度が下がる・反復法を用いて計算してるとき解が収束しないなど, 問題が各所で発生してしまう. 実際に, ロケットの軌道計算や飛行機などの機体の安全性をシミュレーションする際も, 誤差が生じると安全設計が壊れたり, ミサイルも的確に迎撃ない, といった現実世界でも被害を被ってしまう. したがって, 解の精度向上は数値解析における最優先課題の一つであると言えよう.

また, 計算速度というのも, 数値解析において重要である. 数値解析が扱う問題は, 人間が手計算でできないものなので, 計算をできる限り速く行わないと, シミュレーションが終わらないのである. 他にも,CPU 時間や電力などのコスト的な面からみても, 計算に時間がかからない方が良いのは明らかである.

本研究では, その中でも多くの現実問題の核心である「連立一次方程式」の解法についての比較・考察を行う.

2　連立一次方程式とは

連立１次方程式とは, 数学的に一般化して表すと下記のようになるものである.

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
a_{31}x_1 + a_{32}x_2 + \cdots + a_{3n}x_n &= b_3 \\
&\cdots \\
a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n &= b_i \\
&\cdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
\end{aligned}$$

これを数値解析の世界では, 行列を用いて

$$Ax = b$$

と表す.

それぞれ,

- A: 係数行列
- x: 未知数
- b: 定数

を意味する.

一次とは, いわゆる線形のことを指し, 未知数 x に二乗や三乗が含まれていないということである. 実は世の中の現象の多くは「非線形」で, 解くのが非常に困難だ. しかし, 数値解析という分野では, 非線形の問題を「大量の連立一次方程式」に分割して解く手法が一般的になっている. そのため, 物理シミュレーションや機械学習, 検索エンジンなど現実世界の多くの根幹には連立１次方程式を解くということが関連しているのだと分かる.

連立１次方程式を解く方法として, 大きく分けて直接解法と反復解法が挙げられる. 本研究では, ２つの解法がどう進められていくのか, また, 計算速度と精度という面からそれぞれどのような違いがみられるのか, 比較評価・考察をしていくこととする.

3　直接解法

まずはガウスの消去法に代表される直接解法について扱う.

反復法が, 高精度の近似値が得られるまで手順を反復するという手法なのに対し, 直接解法は手順を 1 回実行したら解が求まるという性質がある. これが, 直接解法と呼ばれる所以である.

3.1　ガウスの消去法

ガウスの消去法とは, 最も基本的な直接法である.

まずは簡単な方程式で一見分かりやすいものから扱う. 以下のような連立 2 元 1 次方程式を解くとする.

$$\begin{cases} 2x + 3y = 8 \\ 4x + 10y = 28 \end{cases}$$

これは一般的に中学校で学習する連立方程式の加減法で簡単に解くことができるが, ここではガウスの消去法への理解につながるような解き方で進めていくこととする.

まず前進消去と呼ばれる作業を行う. この問題においては 1 行目を 2 倍して,2 行目から引き方程式を上三角形式にする.

$$\begin{cases} 2x + 3y = 8 \\ 0x + 4y = 12 \end{cases}$$

次に後退代入と呼ばれる動作を行う. $y = 3$ は明らかであるため, この値を上の式に代入する. よって, 解が $(x, y) = (-0.5, 3)$ と求まった.

このように, 前進消去と後退代入という処理により, 上三角形型の方程式へと変形し解を求めるのがガウスの消去法である.

これを一般化し, 以下のような, 実数の係数をもつ n 元連立 1 次方程式を解いていく.

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \qquad (1)\\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \qquad (2)\\
a_{31}x_1 + a_{32}x_2 + \cdots + a_{3n}x_n &= b_3 \qquad (3)\\
&\cdots\\
a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n &= b_i \qquad (i)\\
&\cdots\\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \qquad (n)
\end{aligned}$$

まず, 以下のように (1) 以外の未知数 x_1 を消去する.

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} & b_n \end{pmatrix} \xrightarrow[i\text{ 行}-(l_{i1}\times 1\text{ 行})]{l_{i1}=\frac{a_{i1}}{a_{11}}} \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ 0 & a'_{22} & \dots & a'_{2n} & b'_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & a'_{n2} & \dots & a'_{nn} & b'_n \end{pmatrix}$$

x_1 の消去 ($i = 2, 3, \dots, n$ 行目に対して実行)

- **倍率の算出:**
$$l_{i1} = \frac{a_{i1}}{a_{11}}$$
- **係数の更新 ($j = 2, \dots, n$):**
$$a_{ij} \leftarrow a_{ij} - l_{i1}a_{1j}$$
- **定数項の更新:**
$$b_i \leftarrow b_i - l_{i1}b_1$$

なお, 倍率を掛けて更新した値には $'$ をつけている.

続いて,x_2 の消去を行う.

x_2 の消去 ($i = 3, \dots, n$ 行目に対して実行)

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} & b_1 \\ 0 & a_{22} & a_{23} & \dots & a_{2n} & b_2 \\ 0 & a_{32} & a_{33} & \dots & a_{3n} & b_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & a_{n2} & a_{n3} & \dots & a_{nn} & b_n \end{pmatrix} \xrightarrow[i\text{ 行}-(l_{i2}\times 2\text{ 行})]{l_{i2}=\frac{a_{i2}}{a_{22}}} \begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} & b_1 \\ 0 & a_{22} & a_{23} & \dots & a_{2n} & b_2 \\ 0 & 0 & a'_{33} & \dots & a'_{3n} & b'_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & a'_{n3} & \dots & a'_{nn} & b'_n \end{pmatrix}$$

このようにして,x_3 以降に対する消去を $n-1$ 回目まで行う ($k = 1, 2, \dots, n-1$)

1. **ピボットの選択:** a_{kk} を軸とする。
2. **倍率の計算:** $i = k+1, \dots, n$ 行目に対して
$$l_{ik} = \frac{a_{ik}}{a_{kk}}$$

3. **要素の更新:** $j = k+1, \ldots, n$ 列目に対して

$$a_{ij} \leftarrow a_{ij} - l_{ik} a_{kj}$$

$$b_i \leftarrow b_i - l_{ik} b_k$$

なぜ n 回繰り返すのではなく,$n-1$ 回で終わる理由として,

- **1 回目 (x_1 の消去)**：2 行目から n 行目までを処理して,x_1 を消す.
  （結果：2 行目以降は x_2 から始まる式になる）
- **2 回目 (x_2 の消去)**：3 行目から n 行目までを処理して,x_2 を消す.
  （結果：3 行目以降は x_3 から始まる式になる）
- $\ldots$
- $n-1$ **回目 (x_{n-1} の消去)**：最後の n 行目にある x_{n-1} を消す.
  （結果：n 行目は x_n だけの式になる）

と,$n-1$ 回目の操作が終わった時点で,n 行目はすでに以下の形になっている.

$$a_{nn} x_n = b_n$$

これは, すでに未知数が 1 つだけの状態である.

もしここで n 回目の消去を行おうとしても, 消去すべき相手がいないので, ここで前進消去は自動的に終了となる.

前進消去完了後の上三角行列は

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \ldots & a_{1n} \\ 0 & a_{22} & a_{23} & \ldots & a_{2n} \\ 0 & 0 & a_{33} & \ldots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}$$

のように変形させることができた.

次に, 前進消去が完了した後, 後退代入と呼ばれる処理を行う.

前進消去により得られた上三角行列を用いて、未知数 $x_n, x_{n-1}, \ldots, x_1$ の順に解を求めていく。

上三角行列の各行の方程式は一般に次のように表される：

$$a_{ii} x_i + \sum_{j=i+1}^{n} a_{ij} x_j = b_i$$

これを x_i について解くと, 以下の更新式が得られる：

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^{n} a_{ij} x_j \right) \quad (i = n, n-1, \ldots, 1)$$

1. **最下行 $(i = n)$ の計算**: 未知数が x_n だけなので, 直接解が求まる.

$$x_n = \frac{b_n}{a_{nn}}$$

2. **下から上への代入 $(i = n-1, \ldots, 1)$**: 既に求まった $x_{i+1}, \ldots, x_n$ の値を式に代入し,x_i を順次確定させていく.

3.2 連立１次方程式をガウスの消去法で解くプログラム

下記に, 連立１次方程式をガウスの消去法で解くプログラムを示す.

Python はライブラリを用いたら, コードが実装しやすく小規模の行列に対しては高速であるというメリットがある.

しかし今回は, 手法のアルゴリズムに焦点を置き説明を加えたため, 行の変形やループを自分の手で書くことでコンピュータがどう計算しているのかというアルゴリズムが理解しやすい,c 言語での実装を試みた.

```
#include<stdio.h>

#define N 3

void gauss(double a[][N],double b[],double x[])
{
    double l,m;

    int i,j,k;

    //前進消去
    for (i=0;i<N-1;i++)//基準の行を決める
    {
        for (j=i+1;j<N;j++)//i 行目の下の行を消す
        {
            l=a[j][i]/a[i][i];//何倍して消すかの倍率を計算

            for (k=i+1;k<N;k++)
            {
                a[j][k]=a[j][k]-l*a[i][k];//行の値の更新
            }

            b[j]=b[j]-l*b[i];//係数部分も同様に処理する
        }
    }

    //後退代入
    x[N-1]=b[N-1]/a[N-1][N-1];//一番下の式から最後の変数の値を求める

    for (i=N-2;i>=0;i--)//n-1 行目から 1 行目に向かって下から上へ順番に処理を移していく
    {
        m=0;
        for (j=i+1;j<N;j++)//すでに判明している x の値を式に代入して合計値を計算
        {
            m+=a[i][j]*x[j];
        }
```

```
        x[i]=(b[i]-m)/a[i][i];//解の値を求める
    }
}

int main (void)
{
    double a[N][N]=
    {
        {2,4,-2},
        {3,-2,1},
        {-2,-2,3}
    };

    double b[N]={8,8,-1},x[N];
    int i;

    gauss(a,b,x);

    printf("解は以下のようになる\n");

    for (i=0;i<N;i++)
    {

        printf("x[%d]=%10.1f\n",i,x[i]);
    }

    return 0;
}
```

実行結果は,

$$x[0] = 3.0$$
$$x[1] = 2.0$$
$$x[2] = 3.0$$

と出力された.

3.3 ガウスの消去法の計算量

アルゴリズムの良し悪しを評価する際, 計算にかかった時間で比較するとする. しかし, 実行時間というのは,CPU やメモリなど計算機の性能によっても影響されてしまうのだ. そのため, 純粋にそのアルゴリズムが速いのか否か, 比較することができなくなってしまう. そこで, 計算機の性能に左右されず, さらにデータサイズの増加に対してアルゴリズムがどう変化するかも判明する, 計算量という概念を取り入れることとする.

データ数を n とし,$O(n)$ ならデータ数が増えると処理時間も比例して増加するということを表現する. 例えば, 二重ループを考えると,$n \times n$ 回処理が繰り返されるため, 計算量は $O(n^2)$ となる.

また, $f(n) = 5n^3 + 3n^2 + 1,\ g(n) = n^3$ とおくと,

$$\frac{f(n)}{g(n)} = 5 \quad (n \to \infty)$$

となる. よって $5n^3 + 3n^2 + 1 = O(n^3)$ と表される.

ここで, ガウスの消去法の計算量を考える.

3 重ループの最も内側にあるループにおいては, 減算 1 回, 乗算 1 回が行われており, また, 繰り返しの回数は $(N-1) - i$ 回である. したがって, この for 文では $\{(N-1) - i\} \times 2$ 回の演算処理が行われる.

さらに,

$$m = a[j][i]/a[i][i]$$

$$b[j] = b[j] - m \times b[i]$$

など, 中間ループでは for 文以外に 3 回の演算が行われている. よって, 一番外側のループで見れば, ループ以外は行われていない. 以上より, 全演算回数を求めると：

$$\begin{aligned}
&\sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} \{(2N - 2i - 2) + 3\} \\
&= \sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} (-2i + 2N + 1) \\
&= \sum_{i=0}^{N-2} \{(N-1) - i\}(-2i + 2N + 1) \\
&= 2\sum_{i=0}^{N-2} i^2 + (1 - 4N)\sum_{i=0}^{N-2} i + \sum_{i=0}^{N-2} (2N^2 - N - 1) \\
&= \frac{1}{3}(2N^3 - 9N^2 + 13N - 6) + \frac{1}{2}(-4N^2 + 13N^2 - 11N + 2) + (2N^3 - 3N^2 + 1) \\
&= \frac{2}{3}N^3 + \frac{1}{2}N^2 - \frac{7}{6}N
\end{aligned}$$

次に, 後退代入の 2 重ループについても同様に計算量を求めると, 以下のようになる.

$$\begin{aligned}
&\sum_{i=0}^{N-2}\left\{\left(\sum_{j=i+1}^{N-1}2\right)+2\right\}\\
&=\sum_{i=0}^{N-2}(-2i+2N)\\
&=-2\sum_{i=0}^{N-2}i+\sum_{i=0}^{N-2}2N\\
&=-(N-2)(N-1)+2N(N-1)\\
&=N^2+N-2
\end{aligned}$$

以上より, ガウスの消去法全体の演算回数は, これらを合計して

$$\frac{2}{3}N^3+\frac{3}{2}N^2-\frac{1}{6}N-2$$

となる.

よって, ガウスの消去法の計算複雑度が $O(n^3)$ と表されることが理論的に証明できた. 行列のサイズ N が大きくなったとき, 計算時間は N の 3 乗に比例して爆発的に増えるということを意味する. そのため, 大規模な計算にはガウスの消去法は向いていないと分かる.

4 反復解法

4.1 ヤコビ法

続いて, 反復解法の中でもヤコビ法という手法を扱う.

直接解法同様, 下記のような連立 1 次方程式を考えていく.

$$\begin{aligned}
a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n &= b_1 \quad (1)\\
a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n &= b_2 \quad (2)\\
a_{31}x_1+a_{32}x_2+\cdots+a_{3n}x_n &= b_3 \quad (3)\\
&\cdots\\
a_{i1}x_1+a_{i2}x_2+\cdots+a_{in}x_n &= b_i \quad (i)\\
&\cdots\\
a_{n1}x_1+a_{n2}x_2+\cdots+a_{nn}x_n &= b_n \quad (n)
\end{aligned}$$

ヤコビ法は, 連立一次方程式 $Ax=b$ を解くための反復解法の一つである. 係数行列 A を対角成分 D とそれ以外（下三角成分 L、上三角成分 U）に分割して考える.

行列 A を次のように分解する：

$$A=D+L+U$$

ここで,D は対角行列,L は狭義下三角行列,U は狭義上三角行列である. これを用いて $Ax = b$ を書き換えると：

$$(D + L + U)x = b$$

$$Dx = b - (L + U)x$$

D が正則（対角成分がすべて非零）であれば, 次の反復式が得られる：

$$x^{(k+1)} = D^{-1}\{b - (L + U)x^{(k)}\}$$

実装において, 第 i 成分の更新は以下の数式で行われる：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right) \quad (i = 1, 2, \ldots, n)$$

ここで,$x_j^{(k)}$ は前ステップの解を用い, すべての成分を計算し終えてから一斉に新しい解 $x^{(k+1)}$ に更新するのがヤコビ法の特徴である.

ヤコビ法が収束するための十分条件の一つは, 行列 A が対角優位行列であることである：

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (\text{すべての } i \text{ に対して})$$

これは対角成分の絶対値が, その行の他の成分の絶対値の和よりも大きいことを意味する.

4.2 連立１次方程式をヤコビ法で解くプログラム

以下に連立１次方程式をヤコビ法で解くプログラムコードを示す.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define N 5 //次元数
#define EPS 1e-10 //収束判定する
#define MAX_ITER 300 //最大反復回数

double vector_norm_max(double v[], int n) //ベクトルの最大ノルムを計算する関数
{
    double max = fabs(v[0]);//最初の最大値

    for(int i = 1; i < n; i++)//2 番目以降のときの大小関係を比較する
    {
        if(fabs(v[i]) > max)
        {
            max = fabs(v[i]);//比較して大きい方を返す
        }
    }

    return max;//最大値を返す
}

void jacobi(double a[][N], double b[], double x[])//ヤコビ法
{
    double x_new[N];//新しい近似値
    double diff[N];//差を取る
    double s, eps;//総和と誤差

    int i, j, iter = 0;//iter は反復回数

    do //誤差が十分小さくなるまで繰り返す
    {
        for(i = 0; i < N; i++)//変数を更新
        {
            s = 0.0;
```

```
        for(j = 0; j < i; j++)//ai1 □ X1 □ + □ +ai,i − 1 □ Xi − 1 □
        {
            s += a[i][j] * x[j];
        }

        for(j = i + 1; j < N; j++)//ai,i+1 □ Xi+1 □ + □ +ain □ Xn □
        {
            s += a[i][j] * x[j];
        }

        x_new[i] = (b[i] - s) / a[i][i];//新しい値出力
    }

    for(i = 0; i < N; i++)
    {
        diff[i] = x_new[i] - x[i];//差を求める
    }

    eps = vector_norm_max(diff, N);//誤差を求める

    if(iter >= MAX_ITER)
    {
        printf("収束しませんでした.\n");
        exit(1);
    }

    for(i = 0; i < N; i++)
    {
        x[i] = x_new[i];//解の値の更新
    }

    iter++;

} while(eps > EPS);

printf("解は以下の通り\n");
printf("反復回数: %d 回\n", iter);
```

```
    for(i = 0; i < N; i++)
    {
        printf("x[%d] = %lf\n", i, x[i]);//近似解を表示
    }
}

int main(void)
{
    double a[5][5] = {
        {10, -1,  2,  0,  0},
        {-1, 11, -1,  3,  0},
        { 2, -1, 10, -1,  0},
        { 0,  3, -1,  8, -2},
        { 0,  0,  0, -2,  5}
    };

    double x[5] = {
        0, 0, 0, 0, 0
    };

    double b[5] = {
         6,
        25,
       -11,
        15,
        -6
    };

    jacobi(a, b, x);

    return 0;
}
```

実行結果は,
反復回数: 36 回

$$x[0] = 1.010971$$
$$x[1] = 2.068261$$
$$x[2] = -1.020722$$
$$x[3] = 0.746457$$
$$x[4] = -0.901417$$

と出力された.

4.3 ヤコビ法の計算量

1 回の反復では, その以外の掛け算が $n-1$ 回, 足し合わせるのに $n-2$ 回, 引き算と割り算が 1 回ずつより, 変数 1 つにつきおよそ $2n$ 回くらいの計算回数となる.

ここで, 変数は n 個存在するため, 合計の計算回数は

$$2n * n = 2n^2$$

となり, 計算量は $O(n^2)$ と表される.

よって, ヤコビ法は 1 反復あたりの計算量が $O(N^2)$ と軽量であり, 少ない反復回数 k で収束するような問題（対角優位性が強い行列など）においては, ガウスの消去法の $O(N^3)$ よりも効率的に解を求めることができると分かる.

5 直接解法・反復解法の比較評価

ガウスの消去法に代表される直接解法は, 必要な手順を実行したら必ず解が得られ, 反復法のように解が収束しないというリスクを考慮する必要がないという点が長所として考えられる.

しかし, ガウスの消去法の計算量により, 未知数が 10 倍になれば計算コストは 1000 倍に膨れ上がるということがわかった. さらに, 元の行列が疎行列であっても, 消去の過程で 0 だった場所に値が入るフィルインが発生し, メモリを激しく消費してしまう. また, 無限小数を扱えずどこかで桁を切り捨てるため, 大規模になればなるほど, 丸め誤差が蓄積され, 正確な値からは遠ざかってしまう恐れがある. そのため, 直接解法は大規模な行列を扱う処理は向いていないと考えられる.

対して, 反復解法では誤差がいくつ以下になったら終了と誤差の範囲を自分で設定できる, 0 要素を保存する必要がなくメモリ消費を抑えられる, などの長所が存在する.

しかし, 反復解法には解が収束するか否かは行列の条件数や数学的性質に強く依存し, 条件が悪い場合には解が発散, あるいは収束に膨大な時間を要するという不安定さが短所として考えられる.

6　最後に

本研究では, 物理シミュレーションやデータ解析の根幹をなす「連立一次方程式」の解法に焦点を当てた. 有限回の手順で厳密解を得る「直接解法」と, 逐次近似によって解を追い求める「反復解法」について, それぞれの数学的アルゴリズムを述べ, その上で, 計算コストと精度という二つの観点から両手法を比較評価し, 考察を行った.

直接解法と反復解法には一長一短があり, それぞれの苦手分野や得意なことも分かった. しかし,LU 分解やコレスキー分解,SOR 法など, 取り挙げていない手法もある.

今後, 本研究で扱うことのできなかった手法のアルゴリズムを述べ実装するとともに, 直接解法と反復解法, どちらも取り入れた「ハイブリッド解法」について考えたい. 実際に,LU 分解＋反復解法という処理は, 大規模問題を解く際に使われている.

発想としては, 最初は直接解法という知見で, 前処理をする. 続いてその結果を初期値にして反復解法で精度を上げる, と組み合わせる.

そして, これらの高度なソルバーを音声処理や画像処理における「ノイズ除去」といった実社会の課題へと応用していくことが, 今後への展望である. 例えば, 画像復元は数百万次元におよぶ巨大な逆問題を解く作業だが, ここにハイブリッド解法を適用することで, エッジの鋭さを維持した高精度な復元と, リアルタイムな処理速度を両立できる可能性がある. 理論としての数値解析を, 視覚や聴覚の質を向上させる実践的な技術へと昇華させていきたい.

7　参考文献

・相馬隆郎,”C 言語で作って学ぶ数値計算プログラミング”, 講談社,2025 年

・数値計算の基礎 https://psymonmarkrine.github.io/numerical_computation_kawa7/03.html